ResourceX – Project Contribution Report

Asset & Resource Allocation Management System | Multi-Tenant SaaS Platform

STUDENT NAME	ROLL NUMBER	INSTITUTION
J Prem Sahith	S20240010089	IIIT Sri City

1. Executive Summary of Contributions

As a core contributor to the **ResourceX** enterprise asset management platform, my primary responsibilities focused on:

- **React Conversion of the Registrar Portal:** End-to-end migration of the legacy Registrar interface into a modern, decoupled React 19 architecture featuring synchronized tab routing, reactive state management, asynchronous backend integration, floating feedback notifications, and embedded file inspection.
- **Landing Page Architecture & Conversion (Lab 2):** Modularizing and building the responsive, enterprise-grade public landing page in React with lifted state, reusable card components, and an automated `IntersectionObserver` scroll-spy system.

2. Contribution Detail: React Conversion of Registrar Portal

The Registrar is responsible for university-wide procurement approvals, high-level resource tracking, and inter-departmental request oversight. I converted the monolithic `registrar.html` and `registrar.js` into a modular React dashboard suite located in `react-frontend/src/components/dashboards/registrar/`:

File / Component	Type	Description & Technical Implementation
`RegistrarDashboard.jsx`	Master Layout & Router	Coordinates sidebar navigation, route matching across all 4 registrar sub-views, browser history sync (`useLocation` + `useNavigate`), and hosts global modals and toasts.
`RegistrarOverview.jsx`	Default Hub View	Executes parallel data fetching using `Promise.all` across 4 endpoints (summary, requests, procurements, resources). Renders 4 KPI stat cards and top-5 activity tables with quick-switch routing.
`ProcurementApprovalsView.jsx`	Procurement Workflow	Provides one-click approval/rejection workflows for equipment procurement, cost calculation, rejection reasoning prompt, and attaches spec document viewing hooks.
`GlobalRequestsView.jsx`	Overview & Tracking	Global cross-department tracking table. Dynamically fetches departments from the NestJS backend API, provides status filtering and search indexing.

File / Component	Type	Description & Technical Implementation
`SystemAnalyticsView.jsx`	Analytics & Reporting	Calculates asset allocations across departments, visual progress indicators for operational capacity, and real-time approval-to-pending metrics.
`FilePreviewModal.jsx`	Document Viewer	Secure in-browser preview modal converting Base64 data URLs to Blob URLs (`URL.createObjectURL`). Embeds PDFs via `<iframe>` and renders image specs with download capability.
`RegistrarToast.jsx`	UI Notification	Lightweight, auto-dismissing floating alert system supporting Success (green), Warning (amber), and Error (red) states with Google Material Symbols.
`services/registrar.js`	API Service Layer	Asynchronous HTTP service client handling authentication headers, token propagation, and REST communication with NestJS endpoints.

3. Contribution Detail: Landing Page Architecture & Conversion (Lab 2)

Built the complete public-facing presentation layer for ResourceX in `react-frontend/src/components/landing/`, maintaining strict pixel fidelity, responsiveness, and clean component isolation:

Component	Role & Functionality
`LandingPage.jsx`	Top-level container holding static marketing data trees. Implements a **Scroll-Spy** using `useRef` and `IntersectionObserver` to highlight active navigation items as sections scroll into the viewport.
`Navbar.jsx`	Responsive header with animated active-link indicators, mobile hamburger drawer, and direct routing CTA into the authentication portal (`/login`).
`HeroSection.jsx`	High-impact hero banner with enterprise tagline, call-to-action buttons ("Get Started", "Explore Architecture"), and interactive UI dashboard mockup preview.
`DepartmentsSection.jsx`	Showcases organizational hierarchy, budget caps, and structural operational capacity using modular cards.
`UsersSection.jsx`	Highlights multi-tier role access (Requestor, Department Head, Staff & Maintenance) using colored role indicator badges.
`ResourcesSection.jsx`	Displays asset lifecycle workflows: Live Inventory Tracking, Predictive Maintenance tickets, and Procurement pipelines.
`FeatureCard.jsx` & `RoleCard.jsx`	Highly reusable presentational components with custom color theming, hover elevation effects, and Google Material Symbol integration.
`PartnersSection.jsx` & `Footer.jsx`	Enterprise partner trust badges and standardized footer navigation containing legal, terms, and support links.
`landing.css`	Scoped stylesheets providing CSS Grid layouts, glassmorphism accents, subtle micro-animations, and full mobile breakpoints.

4. Git Commits & Pull Request Reference

Key Pull Requests & Git Commits authored by J Prem Sahith:

- `PR #12` – *"prem sahith pull request merge for registrar page"* (Merged to `main`)
- `commit a615717` – *feat(registrar): dynamically fetch departments from backend in GlobalRequestsView*
- `commit 383f37d` – *refactor(registrar): strictly replicate exact 4 views from previous registrar page*
- `commit 9ab9a06` – *feat(registrar): implement React Registrar dashboard, views, and service*
- `commit 6dea6a6` – *feat: Lab 2 – convert landing page to React* (Modular components, scroll-spy, lifted state)

- `commit 27a6eb6` – *fix(sidebar): properly align brand logo and scope landing styles*

5. Engineering Impact

- **Decoupled Client-Server Architecture:** Replaced tightly coupled DOM manipulation and global variables with React hooks (`useState` , `useEffect` , `useCallback` , `useMemo`).
- **Fault-Tolerant UX:** Implemented parallel asynchronous API calls with independent error catch handlers so that partial network issues do not degrade dashboard availability.
- **Document Safety & Usability:** Embedded in-app PDF and invoice document previews with automatic memory reclamation, eliminating external file download friction for the Registrar.
